

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Báo cáo Đồ án môn học

Đề tài: Hashiwokakero

Môn học: Cơ sở trí tuệ nhân tạo

Giảng viên hướng dẫn:	GS. TS. Lê Hoài Bắc	
	Nguyễn Thanh Tình	
Sinh viên thực hiện:	Nguyễn Trọng Doanh	23120004
	Nguyễn Lê Khánh	23120052
	Trịnh Thị Thu Hiền	23120254
	Hồ Thùy Trâm	23120421

Ngày 21 tháng 12 năm 2025



Mục lục

1	Giới thiệu	1
1.1	Tổng quan về bài toán	1
1.2	Mục tiêu đồ án	1
1.3	Thông tin thành viên	2
1.4	Phân công công việc	3
2	Dạng chuẩn hội (Conjunctive Normal Form - CNF)	3
2.1	Xây dựng biến logic và ràng buộc cầu đôi	3
2.2	Không gian của bài toán	4
2.3	Xây dựng ràng buộc các cây cầu không giao nhau	5
2.4	Xây dựng ràng buộc tổng số cầu nối vào đảo bằng giá trị của đảo	6
2.5	Kiểm tra ràng buộc tạo thành một thành phần liên thông	6
2.6	Tiền xử lý các trường hợp kinh nghiệm	6
2.7	Giải thích ý tưởng cài đặt qua ví dụ quá trình xây dựng ràng buộc	7
3	Cài đặt thuật toán	11
3.1	Giải CNF bằng thư viện PySAT	11
3.1.1	Phân tích cài đặt	11
3.1.2	Phân tích triển khai và tối ưu hóa	12
3.2	Thuật toán tìm kiếm A^*	12
3.2.1	Không gian trạng thái	12
3.2.2	Hàm đánh giá chi phí	13
3.2.3	Tích hợp kỹ thuật suy diễn	13
3.2.4	Xử lý ràng buộc liên thông	13
3.2.5	Phân tích độ phức tạp	14
3.2.6	Mã giả và cài đặt	16
3.3	Thuật toán Backtracking	18
3.3.1	Bruteforce	18
3.3.2	Thuật toán DPLL	18
3.3.3	Kỹ thuật Unit propagation	19

3.3.4	Heuristic MOMs	19
3.3.5	Xử lý ràng buộc liên thông bằng kỹ thuật Lazy evaluation và Blocking clauses	20
4	Algorithms Benchmark và So sánh	21
4.1	Kết quả trung bình	21
4.2	Phân tích và So sánh	22
4.2.1	Thời gian thực thi (Execution Time)	22
4.2.2	Bộ nhớ tiêu thụ (Memory Usage)	24
4.3	Kết luận	25

1 Giới thiệu

1.1 Tổng quan về bài toán

Hashiwokakero (hay còn được gọi là Bridges hoặc Hashi) là một trò chơi giải đố logic nổi tiếng được xuất bản bởi Nikoli. Trò chơi này yêu cầu người chơi phải có tư duy chiến lược và khả năng lập kế hoạch cẩn thận để kết nối các hòn đảo được đánh số lại với nhau bằng các cây cầu, tuân thủ một hệ thống các quy tắc nghiêm ngặt.

Trò chơi diễn ra trên một lưới hình chữ nhật không có kích thước chuẩn định trước. Một số ô trên lưới chứa các con số từ 1 đến 8, đại diện cho các "hòn đảo". Các ô còn lại là ô trống. Mục tiêu của trò chơi là vẽ các cây cầu nối giữa các hòn đảo sao cho thỏa mãn các điều kiện sau:

- Các cây cầu phải bắt đầu và kết thúc tại các hòn đảo riêng biệt, đi theo đường thẳng.
- Cầu chỉ được chạy theo phương vuông góc (ngang hoặc dọc).
- Cầu không được cắt qua các cây cầu khác hoặc các hòn đảo khác.
- Giữa hai hòn đảo bất kỳ chỉ được có tối đa hai cây cầu.
- Số lượng cầu kết nối với mỗi hòn đảo phải bằng đúng con số ghi trên hòn đảo đó.
- Tất cả các hòn đảo phải được kết nối với nhau thành một nhóm liên thông duy nhất.

Với chiều sâu về mặt logic, Hashiwokakero không chỉ là một trò chơi giải trí mà còn là một bài toán thú vị trong lĩnh vực khoa học máy tính, đặc biệt là trong các bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problems - CSP).

1.2 Mục tiêu đồ án

Đồ án này tập trung vào việc áp dụng các thuật toán để giải bài toán Hashiwokakero dưới dạng **Logic mệnh đề (Propositional Logic)**. Cụ thể, các quy tắc của trò chơi sẽ được chuyển đổi thành tập hợp các mệnh đề ở **Dạng chuẩn tắc hội (Conjunctive Normal Form - CNF)**.

Các phương pháp được thực hiện bao gồm:

1. **Sử dụng thư viện PySAT:** Sử dụng thư viện pySAT để tìm lời giải nhanh chóng và làm cơ sở (benchmark) cho các thuật toán tự cài đặt.

2. **Thuật toán tìm kiếm A* (A-Star Search):** Cài đặt thuật toán A* để giải quyết bài toán SAT, sử dụng hàm heuristic dựa trên số lượng mệnh đề chưa thỏa mãn.
3. **Backtracking (DPLL):** Hiện thực thuật toán quay lui kết hợp với các kỹ thuật suy diễn logic như Lan truyền đơn vị (Unit Propagation).
4. **Brute-force:** Cài đặt thuật toán vét cạn trên không gian các biến logic.

Báo cáo này sẽ trình bày chi tiết về quá trình chuyển bài toán sang CNF, chi tiết cài đặt của từng thuật toán giải CNF kể trên, phân tích độ phức tạp và so sánh kết quả thực nghiệm trên các bộ dữ liệu kiểm thử.

1.3 Thông tin thành viên

STT	Họ và tên	MSSV	Đóng góp
1	Nguyễn Trọng Doanh	23120004	25%
2	Nguyễn Lê Khánh	23120052	25%
3	Trịnh Thị Thu Hiền	23120254	25%
4	Hồ Thùy Trâm	23120421	25%

1.4 Phân công công việc

Thành viên	Nội dung công việc	Hoàn thành
Nguyễn Trọng Doanh	• Cài đặt mã nguồn: Hashikowakero, AStarSolver • Sinh test • Viết báo cáo: A*, thực nghiệm • Hoàn thiện, quản lý mã nguồn	100%
Nguyễn Lê Khánh	• Cài đặt mã nguồn: Hashikowakero, Backtracking, Bruteforce • Sinh test • Viết báo cáo: CNF, Backtracking • Hoàn thiện, quản lý mã nguồn	100%
Trịnh Thị Thu Hiền	• Cài đặt mã nguồn: PySATSolver, Backtracking, Bruteforce • Kiểm thử mã nguồn • Viết báo cáo: Backtracking, PySAT • Kiểm tra, hoàn thiện báo cáo	100%
Hồ Thùy Trâm	• Cài đặt mã nguồn: PySATSolver, AStarSolver • Kiểm thử mã nguồn • Viết báo cáo: A*, PySAT • Kiểm tra, hoàn thiện báo cáo	100%

2 Dạng chuẩn hội (Conjunctive Normal Form - CNF)

2.1 Xây dựng biến logic và ràng buộc cầu đôi

Với mỗi câu đố được biểu diễn trên một ma trận có kích thước $R \times C$, trạng thái của bài toán được đặc trưng bởi một tập các biến logic biểu diễn sự tồn tại của một cây cầu đơn hợp lệ nối giữa hai đảo liên tiếp theo phương ngang hoặc phương dọc. Một cây cầu đôi không tồn tại độc lập mà được

mã hóa bằng hai biến cầu đơn và một mệnh đề ràng buộc thứ bậc

$$e_{u,v}^{(2)} \implies e_{u,v}^{(1)}$$

trong đó, u là đảo có vị trí xếp trước v theo thứ tự duyệt từ trái qua phải, từ trên xuống dưới và $e_{u,v}^{(1)}, e_{u,v}^{(2)}$ là hai cây cầu đơn duy nhất nối giữa chúng. Theo cách này, câu đố ban đầu được chuyển thành một đồ thị vô hướng $G = (V, E)$, trong đó tập đỉnh V tương ứng với các đảo và tập cạnh E tương ứng với các biến logic biểu diễn những cây cầu đơn.

2.2 Không gian của bài toán

Trong quá trình tìm hiểu, nhóm chưa tìm thấy bất kỳ nghiêm cứu nào chứng minh về không gian thực tế của bài toán. Một số nhận xét được rút ra như sau:

- Vì mỗi đảo có tối đa 8 cây cầu đơn nối đến (4 cặp cầu đơn nối từ 4 hướng) nên suy ra $|E| \leq \frac{8|V|}{2} = 4|V|$.
- Để nối được tập V đỉnh thành một thành phần liên thông cần ít nhất $|V| - 1$ cạnh, nghĩa là có ít nhất $|V| - 1$ ô trong ma trận ban đầu là ô trống. Từ đó suy ra $|V| + |V| - 1 \leq RC \Leftrightarrow |V| \leq \frac{RC+1}{2}$ hay $|V| \leq \lceil \frac{RC}{2} \rceil$

Từ hai nhận xét trên, ta suy ra được $|E| \leq 2RC$. Thực tế thì E và V không thể hiện mối quan hệ tuyến tính với nhau. Khi số lượng đỉnh đạt giá trị lớn nhất, ma trận sẽ dày đặc (các đảo nằm sát nhau) khiến cho bậc của đỉnh giảm đi. Trong khi đó, để số lượng cạnh hay số bậc của các đỉnh đạt giá trị lớn nhất, các đỉnh phải phân bố cách đều nhau trên ma trận, làm giảm số lượng đảo tối đa trên ma trận. Một số ví dụ cho trường hợp trên (lưu ý $|E|$ là số lượng cạnh không phải số cây cầu):

Bảng 1: Kích thước 5x5 với $|V| = 11$ và $|E| = 22$

2	=	4	=	4
4	=	=	4	\$
\$			\$	4
\$	2	=	4	\$
4	=	4	=	4

Bảng 2: Kích thước 5x5 với $|V| = 9$ và $|E| = 24$

4	=	6	=	4
\$		\$		\$
6	=	8	=	6
\$		\$		\$
4	=	6	=	4

Bảng 3: Kích thước 5x7 với $|V| = 15$ và $|E| = 32$

2	=	4	=	4	=	4
4	=	4	=	=	4	\$
\$					\$	4
\$	2	=	4	=	4	\$
4	=	4	=	4	=	4

Bảng 4: Kích thước 5x7 với $|V| = 12$ và $|E| = 34$

4	=	6	=	6	=	4
\$		\$		\$		\$
6	=	8	=	8	=	6
\$		\$		\$		\$
4	=	6	=	6	=	4

Tuy có thể chỉ ra rằng $|E| \leq 2RC$ nhưng không gian bài toán phụ thuộc chính vào *số lượng hòn đảo* và phân bố của chúng chứ không phải kích thước ma trận. Hơn nữa, ta luôn thấy rằng $|E| \in O(|V|)$ nên không gian tìm kiếm $2^{|E|}$ tăng theo hàm mũ của số lượng đỉnh trong đồ thị.

2.3 Xây dựng ràng buộc các cây cầu không giao nhau

Với mỗi cặp cây cầu đơn thứ nhất $e_{x,y}^{(1)}$, $e_{u,v}^{(1)}$ giao nhau, chúng ta thêm ràng buộc

$$\neg e_{x,y}^{(1)} \vee \neg e_{u,v}^{(1)}$$

nghĩa là "có ít nhất một cây cầu không tồn tại". Vì đã thêm ràng buộc cầu đôi nên không cần thêm ràng buộc không giao nhau cho cây cầu đơn thứ hai $e_{x,y}^{(2)}$, $e_{u,v}^{(2)}$. Trong trường hợp một cây cầu đơn $e_{x,y}^{(1)}$ cắt nhiều cây cầu đơn khác, ràng buộc lớn ban đầu là

$$\neg e_{x,y}^{(1)} \vee \neg(e_{u_1,v_1}^{(1)} \vee e_{u_2,v_2}^{(1)} \vee \dots \vee e_{u_i,v_i}^{(1)})$$

trong đó $e_{u_i, v_i}^{(1)}$ là các cây cầu có phương vuông góc với phương của $e_{x, y}^{(1)}$ và cắt $e_{x, y}^{(1)}$. Tuy nhiên, do chuyển thành dạng chuẩn hội nên ta buộc phải tạo lần lượt i cặp ràng buộc giữa chúng.

2.4 Xây dựng ràng buộc tổng số cầu nối vào đảo bằng giá trị của đảo

Để đơn giản, nhóm sử dụng thư viện hàm `card.CardEnc.equals()` của thư viện PySAT để hỗ trợ với mặc định là kỹ thuật sequential-counter (SC) encoding [Sinz, 2005]. Cơ chế đằng sau để xây dựng ràng buộc bằng là kết hợp hai ràng buộc "bé hơn bằng" và "lớn hơn bằng" với nhau. Kỹ thuật SC tạo thêm $O(nk)$ biến mới, với $n = V$ là số lượng đảo và $k \leq 8$ là giá trị của đảo, đem lại một lựa chọn đơn giản và phù hợp cho các câu đố có kích thước nhỏ.

2.5 Kiểm tra ràng buộc tạo thành một thành phần liên thông

Cấu trúc dữ liệu **Disjoint Set Union** (DSU) [Galil and Italiano, 1991] được cài đặt và sử dụng cho bước kiểm tra tính liên thông. Với độ phức tạp thời gian là $O(\text{ackermann_inverse}(V))$ cho mỗi lần thực hiện nối (union) hai đỉnh, tổng độ phức tạp thời gian trong trường hợp tệ nhất chỉ là $O(E * \text{ackermann_inverse}(V))$. Một cách làm đơn giản hơn là duyệt theo chiều sâu (Depth-First Traversal) từ một đỉnh bất kỳ, đánh dấu các đỉnh đã thăm và kiểm tra số đỉnh đã đi qua. Tuy nhiên, đối với các kỹ thuật tối ưu hóa mở rộng như **nén nút để giảm số lượng ràng buộc chặn** (sẽ trình bày ở phần thuật toán đệ quy - **Backtracking Algorithm**), cấu trúc dữ liệu DSU được tận dụng triệt để các đặc trưng dữ liệu vốn có để hỗ trợ cho kỹ thuật này thay vì phải cài đặt thêm hàm và lưu thêm thông tin nếu sử dụng cách làm DFT đơn thuần.

2.6 Tiền xử lý các trường hợp kinh nghiệm

Bên cạnh các ràng buộc chính của bài toán, một số kinh nghiệm thực tiễn được đưa vào trong quá trình xây dựng ràng buộc như sau:

- Nếu số lượng đảo lớn hơn 2 thì 2 đảo có giá trị 1 không thể có *cầu (đơn)* với nhau và 2 đảo có giá trị 2 không thể có *cầu đôi* với nhau.
- Nếu giá trị của đảo bằng 2 lần số hàng xóm của nó thì thực hiện nối toàn bộ các cây *cầu đôi* từ đảo tới các hàng xóm.

- Nếu giá trị của đảo kém hơn 1 so với 2 lần số hàng xóm của nó thì thực hiện nối toàn bộ các cây *cây cầu đơn* từ đảo tới các hàng xóm. *Một ràng buộc khác* có thể thêm vào khi này là có duy nhất một hàng xóm không nối cầu đôi, tuy nhiên trong quá trình thử nghiệm thì ràng buộc này không đem lại hiệu quả nên *được lược bỏ*.

2.7 Giải thích ý tưởng cài đặt qua ví dụ quá trình xây dựng ràng buộc

Khi đã xác định các biến logic là các cây cầu, chúng ta sẽ thực hiện xây dựng các biểu thức xoay quanh các tính chất của chúng. Từ trang web <https://www.puzzle-bridges.com/>, ta chọn một bài toán để xây dựng các ràng buộc ở dạng chuẩn hội (CNF)

Bảng 5: 7x7 Easy Hashi Puzzle ID: 7,909,434

3			5			2
	2				1	
	4		8		4	
3		1				
			2		1	

Qua phân tích mối quan hệ giữa các ràng buộc, ta có các nhận xét cho quá trình cài đặt sau:

- Trên ma trận, mỗi hòn đảo chỉ chứa thông tin về tọa độ và giá trị. Do đó, ta thực hiện biến đảo thành nút trên đồ thị bằng cách **đánh số lại các hòn đảo và lưu thông tin**. Thông tin của hòn đảo được lưu tại mảng **self.islands**.
- Ràng buộc cầu đôi chỉ xảy ra giữa hai cây cầu đơn nối cùng một cặp đỉnh. Vì vậy ta sử dụng các biến một cách khéo léo để dễ dàng tạo ràng buộc. Do các biến là giá trị nguyên dương, ta quy ước **các biến có giá trị lẻ là cầu đơn và các biến giá trị chẵn liền sau chúng là cây cầu đôi**. Các biến logic được tạo lần lượt theo biến **bridge_indices**.
- Ràng buộc cầu không giao nhau cần xác định tọa độ của đảo. Do đó ta tạo **ánh xạ giữa cạnh và cặp đỉnh** rồi từ đó tìm thông tin của đỉnh qua **self.island**. Mảng ánh xạ chính là **self.bridges** lưu một cặp chỉ số hai đỉnh. Để ý, nếu ta đánh số sao cho toàn bộ các cây cầu có phương ngang nằm ở trước một giá trị **split_index** nào đó, việc kiểm tra giao nhau sẽ đơn giản hơn do giảm được việc phải kiểm tra phương của cây cầu. Vì vậy thêm một lần

nữa, thay vì đơn thuần chọn hai giá trị liên tiếp cho một cây cầu, ta thực hiện **đánh số lần lượt các cây cầu theo phương ngang trước**, sau đó tiếp tục với các cây cầu theo **phương dọc**.

- Ràng buộc tổng số lượng cầu nối tới đảo bằng giá trị của đảo được xử lý sử dụng cách mã hóa của thư viện PySAT. Hàm yêu cầu nhận vào một tập hợp các biến logic cũng như giá trị cần mã hóa. Do đó, ta cần **lưu chỉ số của các cạnh nối tới mỗi đỉnh trong đồ thị**. Thông tin này được lưu ở vector **self.edges**, nhưng do biến là giá trị nguyên dương bắt đầu từ 1, còn các cạnh được đánh số từ 0 nên ta phải **cộng thêm 1** khi đẩy vào vector.
- Ràng buộc liên thông sử dụng cấu trúc dữ liệu DSU để kiểm tra tính liên thông. Cấu trúc dữ liệu này sẽ thực hiện nối các đỉnh của các cạnh được chọn. Do kết quả trả về là danh sách các biến tương ứng với chỉ số của cạnh (cộng thêm 1, như đã giải thích ở trên) nên ta chỉ cần sử dụng ánh xạ **self.bridges** đã được tạo để lấy được chỉ số của đỉnh.
- Các trường hợp kinh nghiệm cũng nhờ đó mà dễ dàng thực hiện hơn khi chỉ cần sử dụng **self.islands** để lấy giá trị của đảo và **self.edges** để lấy thông tin các cạnh nối giữa đảo với các đỉnh hàng xóm.

Để dễ hình dung, ta tiếp tục với bài toán đã chọn. Đầu tiên, ta xác định tọa độ (x, y) các hòn đảo theo *thứ tự từ trái qua phải, từ trên xuống dưới*:

- $(0, 0), (0, 3), (0, 6)$
- $(2, 1), (2, 5)$
- $(4, 1), (4, 3), (4, 5)$
- $(5, 0), (5, 2)$
- $(6, 3), (6, 5)$

Với mỗi cặp đảo có cùng tọa độ y và **sai khác tọa độ y lớn hơn 1** (cần lưu ý điều này khi **cài đặt**), ta sử dụng 2 biến liên tiếp logic để biểu diễn cặp cầu đơn:

- 1, 2, 3, 4
- 5, 6

- 7, 8, 9, 10

- 11, 12

- 13, 14

Như vậy, **split_index = 14**, tiếp tục sắp xếp các đảo dựa trên tọa độ theo *thứ tự từ trên xuống dưới, từ trái qua phải*:

- (0, 0), (5, 0)

- (2, 1), (4, 1)

- (5, 2)

- (0, 3), (4, 3), (6, 3)

- (2, 5), (4, 5), (6, 5)

- (0, 6)

Tương tự, với mỗi cặp đảo có cùng tọa độ y và **sai khác tọa độ x lớn hơn 1** (cần lưu ý điều này khi cài đặt), ta sử dụng 2 biến logic liên tiếp để biểu diễn cặp cầu đơn:

- 15, 16

- 17, 18

- Không có

- 19, 20, 21, 22

- 23, 24, 25, 26

- Không có

Vậy là tổng cộng tạo ra 26 biến logic. Tiếp theo, ta thực hiện thêm ràng buộc cầu đôi cho các cặp biến ở dạng chuẩn hội

$$(1 \vee \neg 2) \wedge (3 \vee \neg 4) \wedge (5 \vee \neg 6) \wedge (7 \vee \neg 8) \wedge (9 \vee \neg 10) \wedge (11 \vee \neg 12) \wedge (13 \vee \neg 14) \wedge (15 \vee \neg 16)$$

$$\wedge(17 \vee \neg 18) \wedge (19 \vee \neg 20) \wedge (21 \vee \neg 22) \wedge (23 \vee \neg 24) \wedge (25 \vee \neg 26)$$

Tiếp theo là thêm ràng buộc cầu không cắt nhau, tận dụng việc dữ liệu đã được cấu trúc, ta chỉ cần duyệt qua các biến bé hơn `split_index`, lấy thông tin của cây cầu lưu tại `self.bridges` và kiểm tra với các biến lớn hơn hoặc bằng `split_index` để thêm ràng buộc. Với câu đồ đơn giản đã chọn, kết quả thu được thêm một ràng buộc mới

$$(\neg 5 \vee \neg 19)$$

Các bước thêm ràng buộc tổng số cạnh nối vào đảo phải bằng giá trị của đảo được thực hiện do thư viện PySAT hỗ trợ, do đó nhóm không trình bày ở đây. Lưu ý quan trọng là sau bước này, **các biến phụ sẽ được thêm vào để hỗ trợ tạo ràng buộc.**

Cuối cùng, chúng ta kết thúc quá trình tạo ràng buộc bằng các trường hợp kinh nghiệm. Quan sát câu đồ, ta thấy:

- Câu đồ không tồn tại cặp đỉnh 1 hay cặp đỉnh 2 là hàng xóm của nhau, do đó ta không thêm ràng buộc đỉnh cô lập.
- Đảo (0,0) có giá trị kém hơn 1 so với 2 lần số hàng xóm, ta xây mọi cây cầu đơn nối tới các hàng xóm bằng các ràng buộc

$$1 \wedge 15$$

- Đảo (0,3) tương tự đảo (0,0), ta thêm ràng buộc nối đơn

$$1 \wedge 3 \wedge 19$$

- Đảo (0,6) có giá trị đúng bằng 2 lần số hàng xóm, do đó ta xây mọi cây cầu đôi tới hàng xóm bằng các ràng buộc

$$4$$

Như đã giải thích, vì đã tồn tại ràng buộc cầu đôi nên ta không thêm ràng buộc cầu đơn

$$3$$

- Đảo (4,1) tương tự đảo (0,6), ta thêm các ràng buộc nối đôi

$$8 \wedge 18$$

- Đảo (4, 3) tương tự đảo (0, 6), ta thêm các ràng buộc nối đôi

$$8 \wedge 10 \wedge 20 \wedge 22$$

- Đảo (5, 0) tương tự đảo (0, 0), ta thêm các ràng buộc nối đơn

$$11 \wedge 15$$

- Đảo (5, 2) tương tự đảo (0, 0), ta thêm các ràng buộc nối đơn

$$11$$

Để ý rằng, hai ràng buộc cầu đôi và cầu không giao nhau *không sinh ra ràng buộc trùng*. Tuy nhiên, ba ràng buộc còn lại sẽ có khả năng sinh ra nhiều ràng buộc đã tồn tại trước đó, vì vậy ta cần thực hiện lọc lại ràng buộc. Bước này có thể sử dụng cấu trúc dữ liệu **Set** để hỗ trợ.

3 Cài đặt thuật toán

3.1 Giải CNF bằng thư viện PySAT

Sau khi đã mã hóa các ràng buộc cục bộ (bậc của đỉnh, tính phẳng, cầu đơn/đôi) thành các mệnh đề CNF, thách thức lớn nhất còn lại của bài toán Hashi là tính liên thông toàn cục. Việc mã hóa tĩnh (static encoding) tính liên thông ngay từ đầu thường dẫn đến sự bùng nổ tổ hợp số lượng mệnh đề, khiến bài toán trở nên bất khả thi với lưới kích thước lớn. Để giải quyết vấn đề này, giải pháp sử dụng chiến lược Lazy Constraints kết hợp với khả năng Incremental Solving của bộ giải SAT hiện đại (như Glucose 3/4) có trong thư viện PySAT.

3.1.1 Phân tích cài đặt

Thay vì ép buộc đồ thị phải liên thông ngay lập tức, chúng ta chia quá trình giải thành một vòng lặp lặp lại giữa hai bước: Tìm nghiệm cục bộ và Kiểm tra/Sửa lỗi toàn cục.

1. Giải bài toán nối lỏng: Tìm một nghiệm thỏa mãn các bậc đảo và quy tắc không giao cắt. Kết quả có thể là một rừng các thành phần rời rạc.
2. Xác định xung đột (Lát cắt): Nếu đồ thị không liên thông, DSU sẽ xác định thành phần lớn nhất S và phần còn lại $\bar{S}$.

3. Thêm mệnh đề cắt: Để kết nối S với phần còn lại, ít nhất một cây cầu tiềm năng nối giữa S và $\bar{S}$ phải được kích hoạt. Công thức: $\bigvee_{e \in E_{cut}} x_{e,1f}$
4. Lập lại: Trình giải SAT tiếp tục tìm kiếm với ràng buộc mới này cho đến khi đạt được sự liên thông toàn cục.

3.1.2 Phân tích triển khai và tối ưu hóa

- Vector hóa với Numpy: Sử dụng np.where và np.lexsort để xác định hàng xóm của các đảo thay vì dùng vòng lặp lồng nhau, chuyển đổi độ phức tạp từ $O(N^2)$ sang $O(N \log N)$.
- Quản lý mệnh đề: Loại bỏ các mệnh đề trùng lặp để giảm dấu ấn bộ nhớ và giảm tải cho quá trình lan truyền.
- Incremental Solving: Tận dụng giao diện của thư viện PySAT để giữ lại trạng thái nội bộ của solver (các mệnh đề đã học) giữa các lần lặp của vòng lặp lazy, thay vì khởi động lại từ đầu.

3.2 Thuật toán tìm kiếm A*

Trong khi thuật toán DPLL dựa trên cơ chế tìm kiếm theo chiều sâu DFS để duyệt không gian các biến, thuật toán A* tiếp cận bài toán theo hướng tìm kiếm ưu tiên (Best-First Search). Trong ngữ cảnh của bài toán SAT, mục tiêu của A* không tập trung vào tìm đường đi ngắn nhất theo nghĩa đen, mà hướng đến xác định một trạng thái đích có thể chấp nhận được một cách nhanh chóng dựa trên định hướng từ hàm heuristic.

3.2.1 Không gian trạng thái

Không gian trạng thái của bài toán SAT được định nghĩa tương tự như trong thuật toán Backtracking, nhưng cách quản lý và mở rộng trạng thái có sự khác biệt:

- **Trạng thái (Node):** Một bộ đôi $(\mathcal{A}, \mathcal{U})$, trong đó $\mathcal{A}$ là tập hợp các biến đã được gán giá trị và $\mathcal{U}$ là tập hợp các chỉ số của các mệnh đề chưa được thỏa mãn.
- **Cơ chế quản lý:** Thay vì sử dụng Ngăn xếp (Stack), A* sử dụng Hàng đợi ưu tiên Min-Priority Queue để luôn mở rộng trạng thái có giá trị hàm chi phí $f(n)$ nhỏ nhất.

3.2.2 Hàm đánh giá chi phí

Hàm đánh giá $f(n) = g(n) + h(n)$ được thiết kế để cân bằng giữa chi phí thực tế và ước lượng khoảng cách đến đích, với xu hướng tham lam (Greedy) để nhanh chóng tìm được trạng thái đích:

Chi phí thực tế $g(n)$

$$g(n) = |\mathcal{A}|$$

Đây là số lượng biến đã được gán giá trị, tương ứng với độ sâu của nút trong cây tìm kiếm.

Hàm Heuristic $h(n)$

$$h(n) = |\mathcal{U}|$$

Đây là số lượng mệnh đề CNF còn lại chưa được thỏa mãn. Heuristic này thỏa mãn tính chất *admissible* (chấp nhận được) vì để thỏa mãn k mệnh đề còn lại, cần thực hiện tối thiểu 1 phép gán biến (trong trường hợp tối ưu nhất một biến thỏa mãn tất cả). Do đó $h(n)$ không bao giờ ước lượng vượt quá chi phí thực tế cần thiết.

3.2.3 Tích hợp kỹ thuật suy diễn

Để giải quyết vấn đề bùng nổ không gian trạng thái, thuật toán A^* trong đồ án tái sử dụng các kỹ thuật suy diễn đã được trình bày chi tiết ở phần **Backtracking Algorithm**:

- **Lan truyền đơn vị (Unit Propagation):** Được tích hợp vào bước sinh trạng thái kế tiếp (Successor Generation). Thay vì chỉ gán đơn lẻ một biến và thêm vào hàng đợi, thuật toán chạy Unit Propagation để suy diễn ra một chuỗi các biến bắt buộc khác. Điều này giúp A^* bỏ qua qua nhiều tầng của cây tìm kiếm và loại bỏ sớm các trường hợp mâu thuẫn.
- **Lựa chọn biến phân nhánh:** Sử dụng heuristic MOMs (như đã mô tả ở phần trước) để chọn biến tối ưu nhất cho việc phân nhánh, nhằm kích hoạt Unit Propagation hiệu quả nhất.

3.2.4 Xử lý ràng buộc liên thông

Tương tự như Backtracking, A^* áp dụng kỹ thuật **Đánh giá trễ (Lazy Evaluation)**. Thuật toán tập trung tìm kiếm trạng thái thỏa mãn logic cục bộ ($h(n) = 0$). Chỉ khi đạt được trạng thái này,

module kiểm tra liên thông (sử dụng cấu trúc dữ liệu **DSU** đã trình bày ở phần CNF) mới được gọi để kiểm tra tính hợp lệ của lời giải.

3.2.5 Phân tích độ phức tạp

Hiệu năng của thuật toán A^* giải quyết bài toán SAT phụ thuộc vào kích thước không gian tìm kiếm và chi phí tính toán tại mỗi trạng thái. Chúng ta định nghĩa các tham số phân tích như sau:

- N : Số lượng hòn đảo (đỉnh của đồ thị).
- E : Số lượng biến logic trong bài toán SAT. Mỗi cạnh tiềm năng giữa hai đảo sinh ra 2 biến (cầu đơn/đôi), do đó $E \approx 2 \times$ số cạnh tiềm năng.
- C : Tổng số mệnh đề (clauses) trong công thức CNF.
- M : Số lượng trạng thái tối đa được lưu trữ trong hàng đợi ưu tiên (Open Set).

Mối quan hệ giữa không gian biến E và kích thước đồ thị N Một đặc điểm quan trọng của bài toán Hashiwokakero là tính chất thừa của đồ thị lưới (Grid Graph). Mỗi hòn đảo chỉ có tối đa 4 hàng xóm (theo các hướng Đông, Tây, Nam, Bắc). Theo định lý bắt tay (Handshaking lemma), tổng số cạnh tiềm năng bị giới hạn bởi:

$$\text{Số cạnh tiềm năng} \leq \frac{4N}{2} = 2N$$

Suy ra số lượng biến logic E tăng tuyến tính theo số lượng đảo:

$$E \approx 2 \times (2N) = 4N \implies E \in O(N)$$

Điều này chỉ ra rằng độ phức tạp của bài toán phụ thuộc vào **số lượng hòn đảo** chứ không phụ thuộc vào kích thước ma trận lưới ($R \times C$). Đây là lý do thuật toán có thể giải quyết các bản đồ lớn (20×20) nếu mật độ đảo không quá cao.

Độ phức tạp thời gian Do bài toán SAT thuộc lớp **NP-complete**, không tồn tại thuật toán đa thức để giải quyết (trừ khi $P = NP$).

- **Không gian tìm kiếm:** Trong trường hợp xấu nhất, A^* phải duyệt qua toàn bộ cây nhị phân của các phép gán biến. Độ sâu của cây là E .

$$T_{\text{search}} = O(2^E)$$

- **Chi phí tại mỗi nút:** Tại mỗi bước mở rộng trạng thái, thuật toán thực hiện:
 - *Lan truyền đơn vị:* Cần duyệt qua danh sách các mệnh đề bị ảnh hưởng. Trong trường hợp tệ nhất, chi phí này tỉ lệ thuận với kích thước bộ CNF: $O(C)$.
 - *Tính toán Heuristic:* Đếm số mệnh đề chưa thỏa mãn: $O(C)$.
 - *Thao tác hàng đợi ưu tiên:* Thêm và lấy phần tử từ Heap: $O(\log M)$.
- **Kiểm tra liên thông:** Chỉ thực hiện tại nút lá khi tìm thấy nghiệm ($h(n) = 0$). Sử dụng cấu trúc Disjoint Set Union (DSU), chi phí là $O(E \cdot \alpha(N))$, trong đó α là hàm nghịch đảo Ackermann (gần như hằng số). Chi phí này là không đáng kể so với quá trình tìm kiếm.

Độ phức tạp thời gian tổng quát:

$$O(2^E \cdot (C + \log M))$$

Nhận xét thực nghiệm: Nhờ kỹ thuật Unit Propagation cắt tỉa mạnh các nhánh mâu thuẫn và Heuristic MOMs định hướng tốt, hệ số phân nhánh trung bình thực tế b nhỏ hơn nhiều so với 2 ($b \ll 2$), giúp thời gian chạy thực tế nhanh hơn nhiều so với cận trên lý thuyết.

Độ phức tạp không gian Độ phức tạp không gian là điểm yếu chính của A^* so với các thuật toán Đệ quy (Backtracking) do phải lưu trữ danh sách các trạng thái biên.

- **Lưu trữ dữ liệu tĩnh:** Bộ knowledge base CNF ban đầu tốn $O(C)$.
- **Lưu trữ trạng thái động:** Mỗi trạng thái trong hàng đợi cần lưu:
 - Phép gán hiện tại: Tốn $O(E)$.
 - Danh sách các mệnh đề chưa thỏa mãn: Tốn $O(C)$.

- **Kích thước hàng đợi:** Trong trường hợp xấu nhất, số lượng nút trong Open Set có thể lên tới $M \approx 2^{E-1}$ (lớp đáy của cây tìm kiếm).

Độ phức tạp không gian tổng quát:

$$O(C + M \cdot (E + C))$$

3.2.6 Mã giả và cài đặt

Algorithm 1 A* SAT Search for Hashiwokakero

```

1: Input: cnf_clauses, num_vars
2: Output: Solution to the puzzle or failure
3: initial_assignment  $\leftarrow \emptyset$ , unsatisfied  $\leftarrow$  all clause indices
4: Unit Propagation: assignment, unsatisfied, valid  $\leftarrow$  propagate(initial_assignment, unsatisfied)
5: if not valid then return failure
6: end if
7: Initialize priority queue: open_list  $\leftarrow$  [State(assignment, unsatisfied)]
8: while open_list is not empty do
9:   current  $\leftarrow$  dequeue(open_list)
10:  if current.h == 0 then ▷ All clauses satisfied (SAT Solution)
11:    solution  $\leftarrow$  build_solution(current.assignment)
12:    if check_connectivity(solution) then ▷ Global Graph Constraint
13:      return solution
14:    end if
15:    continue ▷ Valid SAT but disconnected graph, skip
16:  end if
17:  var  $\leftarrow$  select_variable_moms(current) ▷ Heuristic Selection
18:  for all value in {True, False} do
19:    new_assign  $\leftarrow$  current.assignment  $\cup$  {var: value}
20:    temp_unsatisfied  $\leftarrow$  update_clauses(current.unsatisfied, var, value)
21:    final_assign, final_unsatisfied, valid  $\leftarrow$  propagate(new_assign, temp_unsatisfied)
22:    if valid then
23:      Compute heuristic: h  $\leftarrow$  |final_unsatisfied|
24:      Compute cost: g  $\leftarrow$  |final_assign|
25:      Score: f  $\leftarrow$  g + h
26:      Enqueue State(final_assign, final_unsatisfied, g, h) into open_list
27:    end if
28:  end for
29: end while
30: return failure

```

Implementation Details

- **CNFState**: Đại diện cho một nút trong cây tìm kiếm. Lớp này lưu trữ **phép gán hiện tại** (assignment) của các biến logic và một tập hợp các **chỉ số mệnh đề chưa thỏa mãn** (unsatisfied_indices).
- **propagate (Unit Propagation)**: Hàm này thực hiện việc tìm các **Mệnh đề Đơn vị** (Unit Clauses - mệnh đề chỉ còn lại một biến chưa gán). sau đó gán giá trị bắt buộc cho biến đó để thỏa mãn mệnh đề, giúp thu hẹp đáng kể không gian tìm kiếm bằng cách tự động suy diễn các nước đi bắt buộc. Hàm trả về cờ mâu thuẫn (**conflict**) nếu phát hiện sự vi phạm logic, giúp cắt tỉa nhánh ngay lập tức.
- **select_variable_moms**: Cài đặt heuristic **MOMs** (Maximum Occurrences in Minimum length clauses) để chọn biến phân nhánh. Thay vì chọn ngẫu nhiên, hàm ưu tiên chọn biến xuất hiện thường xuyên nhất trong các mệnh đề chưa thỏa mãn có độ dài ngắn nhất nhằm mục đích thỏa mãn các ràng buộc khó hoặc giúp lan truyền/mâu thuẫn sớm nhất có thể.
- **update_clauses**: Hàm cập nhật tăng dần (incremental update). Khi một biến được gán giá trị, hàm này cập nhật hiệu quả tập hợp các mệnh đề chưa thỏa mãn bằng cách loại bỏ những mệnh đề đã trở nên “Đúng” (True) nhờ phép gán mới, tránh việc phải quét lại toàn bộ bộ CNF, giúp giảm chi phí tính toán tại mỗi bước.
- **check_connectivity**: Hàm kiểm tra hợp lệ, chỉ được gọi khi đã tìm thấy một nghiệm SAT thỏa mãn logic (khi $h = 0$). Vì mã hóa CNF chỉ đảm bảo các ràng buộc cục bộ (số câu, không cắt nhau), hàm này dựng lại đồ thị từ các biến câu và sử dụng cấu trúc dữ liệu **Disjoint Set Union (DSU)** để xác minh rằng tất cả các hòn đảo tạo thành một thành phần liên thông duy nhất.

Ở mỗi bước, thuật toán chọn trạng thái có $f(n)$ nhỏ nhất, kiểm tra điều kiện đích, sau đó phân nhánh trên biến được chọn bởi heuristic MOMs và áp dụng Unit Propagation. Unit Propagation giúp tự động suy diễn nhiều phép gán bắt buộc và phát hiện mâu thuẫn sớm, trong khi heuristic MOMs ưu tiên các biến có khả năng kích hoạt suy diễn mạnh.

3.3 Thuật toán Backtracking

3.3.1 Bruteforce

Thuật toán duyệt trâu có mô hình tương tự với quá trình tìm kiếm theo chiều sâu (DFS) trên cây trạng thái của bài toán với cấu trúc gồm:

- Nút gốc: Trạng thái khởi tạo của đồ thị. Mọi biến đều chưa được gán giá trị.
- Các nút trung gian: Đại diện cho trạng thái sau khi thực thi một tập các quyết định.
- Cạnh nối: Đại diện cho hành động gán giá trị cho một biến chưa được gán.

Cơ chế hoạt động: Thuật toán lựa chọn một biến chưa được gán và thử gán một giá trị hợp lệ. Sau đó, nó tiếp tục đệ quy sang biến tiếp theo. Nếu tại bất kỳ bước nào, việc gán giá trị dẫn đến vi phạm ràng buộc, thuật toán sẽ thực hiện quay lui, hủy bỏ phép gán hiện tại và quay lại trạng thái trước đó để thử giá trị khác.

Hạn chế: Bruteforce thuần túy thường gặp phải vấn đề **thrashing**. Ví dụ, một phép gán sai ở độ sâu 5 có thể không gây ra mâu thuẫn ngay lập tức mà chỉ được phát hiện ở độ sâu 20. Thuật toán sẽ phải thử và sai hàng triệu trạng thái trong cây con của độ sâu 5 trước khi nhận ra vấn đề nằm ở phép gán ban đầu.

Phân tích độ phức tạp: Trong lý thuyết, độ phức tạp của Backtracking phụ thuộc vào độ sâu trung bình mà tại đó mâu thuẫn được phát hiện. Nếu mâu thuẫn được phát hiện ở độ sâu $d \ll N$, thuật toán tiết kiệm được 2^{N-d} bước tính toán. Trong trường hợp tệ nhất, độ phức tạp là hàm mũ $O(2^N \cdot M)$ với N là số biến và M là số mệnh đề.

Để tránh tìm kiếm một cách mù quáng, nhóm đã tối ưu thuật toán Backtracking bằng cách tích hợp các kỹ thuật suy diễn và heuristic như DPLL và MOMs để phát hiện mâu thuẫn sớm nhất có thể (Fail-First Principle), giúp cải thiện thời gian thực thi một cách đáng kể.

3.3.2 Thuật toán DPLL

Thay vì sử dụng Backtracking thuần túy, nhóm cài đặt DPLL (Davis–Putnam–Logemann–Loveland) [Davis et al., 1962], một thuật toán tìm kiếm tiêu chuẩn để giải quyết bài toán SAT và CSP. Thuật toán DPLL xây dựng một cây tìm kiếm bằng cách gán giá trị cho từng biến và thực hiện các bước suy diễn logic để cắt tỉa các nhánh không khả thi thông qua quá trình lan truyền đơn vị (Unit Propagation).

Cấu trúc đệ quy của DPLL bao gồm:

1. Unit propagation: Bước suy diễn logic để gán giá trị cho các biến ràng buộc.
2. Conflict detection: Kiểm tra xem trạng thái hiện tại có vi phạm bất kỳ ràng buộc nào không. Nếu có, trả về FALSE và thực hiện quay lui.
3. Splitting / Branching: Nếu không còn suy diễn nào có thể thực hiện được và chưa tìm ra lời giải, chọn một biến (dựa trên Heuristic MOMs) và phân nhánh tìm kiếm.

Quy tắc phân nhánh được dựa trên *định lý khai triển Shannon*:

$$SAT(\phi) \iff SAT(\phi \wedge x) \vee SAT(\phi \wedge \neg x) \quad (1)$$

Trong đó ϕ là công thức biểu diễn trạng thái đồ thị, và x là một biến quyết định.

Điểm khác biệt cốt lõi giữa DPLL và Backtracking thông thường là DPLL thực hiện bước Unit Propagation ngay sau mỗi lần tách nhánh. Nếu thuật toán đệ quy thông thường thực hiện giá trị cho x và tiếp tục gán ngẫu nhiên y , DPLL thực hiện quá trình suy diễn (implication) để xem hành động gán giá trị cho x có dẫn đến các ràng buộc $y, z, w...$ phải nhận giá trị cụ thể nào không, giúp giảm độ sâu của cây tìm kiếm.

3.3.3 Kỹ thuật Unit propagation

Lan truyền đơn vị là kỹ thuật tối ưu hóa nền tảng trong các thuật giải xử lý bài toán SAT. Một mệnh đề ở dạng chuẩn hội được gọi là mệnh đề đơn vị (unit clause) nếu tất cả các biến trong đó trừ một biến đã được xác định giá trị là FALSE. Quá trình lan truyền được lặp lại cho đến khi không còn suy diễn nào khả thi hoặc phát hiện mâu thuẫn. Nếu mâu thuẫn xảy ra, thuật toán DPLL sẽ lập tức quay lui giúp tiết kiệm các bước tính toán vô ích sau đó.

3.3.4 Heuristic MOMs

Khi quá trình lan truyền đơn vị đã dừng lại mà bài toán vẫn chưa được giải quyết hoàn toàn (vẫn còn các biến chưa gán), thuật toán phải thực hiện chọn biến tiếp theo để tiếp tục rẽ nhánh. Nhóm sử dụng heuristic MOMs (Maximum Occurrences in Minimum Sized clauses) [Crawford and Auton, 1996], một heuristic dễ cài đặt và sử dụng để hỗ trợ bước quan trọng này. Sử dụng heuristic này, khi chọn phân nhánh trên biến có độ ưu tiên cao nhất sẽ xảy ra hai trường hợp:

- Trường hợp gán đúng: Ta giải quyết được điểm nghẽn (bottleneck) khó nhất của bài toán, giúp các bước tiếp theo trở nên dễ dàng hơn.
- Trường hợp gán sai: Vì biến nằm trong vùng ràng buộc chặt, mâu thuẫn sẽ nảy sinh gần như ngay lập tức khi thực hiện lan truyền đơn vị, giúp phát hiện sớm để tĩa nhánh.

So với các heuristic khác như DLIS (Dynamic Largest Individual Sum - chỉ đếm số lần xuất hiện) hay VSIDS (Variable State Independent Decaying Sum - dùng trong các solver như MiniSat với cơ chế học mệnh đề), MOMs phù hợp hơn với cấu trúc tĩnh của Hashikowakero vì nó chỉ tập trung vào những ràng buộc sẵn có.

3.3.5 Xử lý ràng buộc liên thông bằng kỹ thuật Lazy evaluation và Blocking clauses

Ràng buộc khó nhất trong Hashiwokakero là đảm bảo các đảo phải tạo thành một thành phần liên thông. Việc mã hóa điều này trực tiếp vào SAT/CSP là rất tốn kém. Vì thế, nhóm giải bằng việc cài đặt phương pháp Lazy Constraints [Stuckey, 2010] kết hợp với Blocking clauses (mệnh đề chặn).

Nếu ta cố gắng liệt kê tất cả các điều kiện liên thông ngay từ đầu, ta phải thêm các ràng buộc dạng: "Không tồn tại tập hợp con các đảo $S \subset V$ sao cho không có cạnh nào nối S với $V \setminus S$ ". Số lượng tập con S là lũy thừa của số đảo, làm cho việc khởi tạo bài toán trở nên bất khả thi với bài toán lớn.

Sử dụng phương pháp Lazy Constraints, ta bỏ qua ràng buộc liên thông toàn cục và chỉ tập trung giải quyết các ràng buộc cục bộ. Quy trình như sau:

1. DPLL tìm ra một phép gán đầy đủ A thỏa mãn tất cả các số trên đảo.
2. Kiểm tra liên thông: Thuật toán tạm dừng và chạy một thuật toán trên cấu hình A để kiểm tra xem đồ thị có liên thông hay không.
3. Nếu liên thông: Trả về kết quả thành công.
4. Nếu không liên thông:
 - Phát hiện các thành phần liên thông rời rạc.
 - Tạo ra một mệnh đề chặn để ngăn thuật toán tiếp tục lặp lại cấu hình.
 - Thêm mệnh đề này vào tập ràng buộc và thực hiện lại giải thuật.

Để đảm bảo tính liên thông, nhóm sử dụng **lý thuyết tập cắt (Cut set)** với phát biểu: Giả sử tại bước kiểm tra, tập các đỉnh V bị chia thành các thành phần liên thông rời rạc. Theo nguyên lý đồ thị, để S không bị cô lập, bắt buộc phải có ít nhất một cạnh nối từ S sang phần còn lại của đồ thị ($V \setminus S$). Đồng thời, nhóm đưa ra nhận xét **cải tiến Greedy Cut Set** để hỗ trợ quá trình thêm ràng buộc chặn.

Cụ thể, khi lựa chọn tập S để sinh ràng buộc, thay vì chọn ngẫu nhiên một hay nhiều tập, ta chọn tập S^* là **thành phần liên thông có kích thước lớn nhất**, giúp giảm số kích thước của ràng buộc chặn. Qua quá trình thực nghiệm, nhận xét đã chứng minh độ hiệu quả khi **giảm số lần chạy lại thuật toán ở các câu đố khó**. Mệnh đề chặn được thêm vào tập CNF có dạng:

$$\bigvee_{e \in \text{Cut}(S^*, V \setminus S^*)} x_e \quad (2)$$

Trong đó x_e là biến logic đại diện cho sự tồn tại của cạnh e nối giữa S^* và $V \setminus S_{max}$.

4 Algorithms Benchmark và So sánh

Trong phần này, hiệu năng của bốn thuật toán (**Bruteforce**, **Backtracking**, **A* Search** và **PySAT**) được đánh giá dựa trên hai tiêu chí chính là thời gian thực thi và bộ nhớ tiêu thụ.

4.1 Kết quả trung bình

Dưới đây là bảng tổng hợp dữ liệu trung bình sau khi chạy các bộ tests.

Bảng 6: Thống kê thời gian thực thi trung bình (đơn vị: s)

Kích thước	Bruteforce	Backtracking	AStar	PySAT
7x7	46.50972	0.01254	0.03796	0.00068
9x9	1128.81300	0.04960	0.15112	0.00140
11x11	TLE	0.89834	1.13762	0.00278
13x13	TLE	0.49720	1.80770	0.00406
17x17	TLE	33.57936	34.19166	0.00594
20x20	TLE	87.94196	84.71434	0.00762

**Bruteforce cho thời gian chạy rất lâu trong các bộ test vừa và lớn, nên không tính vào bảng (TLE).*

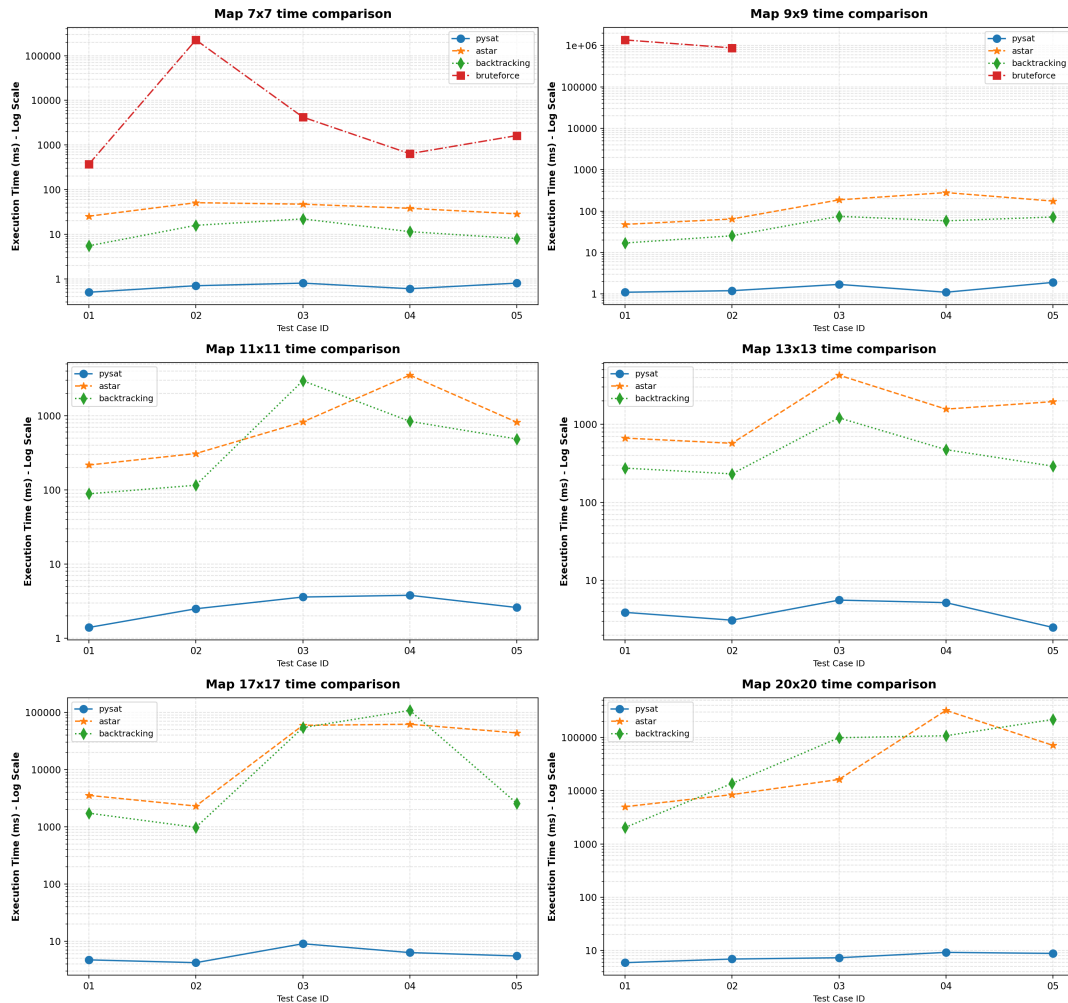
Bảng 7: Thống kê bộ nhớ tiêu thụ trung bình (đơn vị: MB)

Kích thước	Bruteforce	Backtracking	AStar	PySAT
7x7	0.03042	0.09662	0.12306	0.01218
9x9	0.05230	0.28040	0.24314	0.01754
11x11	-	1.05304	0.57378	0.03952
13x13	-	1.89246	0.77396	0.05350
17x17	-	8.65270	1.78824	0.08998
20x20	-	14.50874	3.27720	0.11060

4.2 Phân tích và So sánh

Dựa trên các biểu đồ so sánh (Hình 1 và 2), nhóm đưa ra các nhận xét sau:

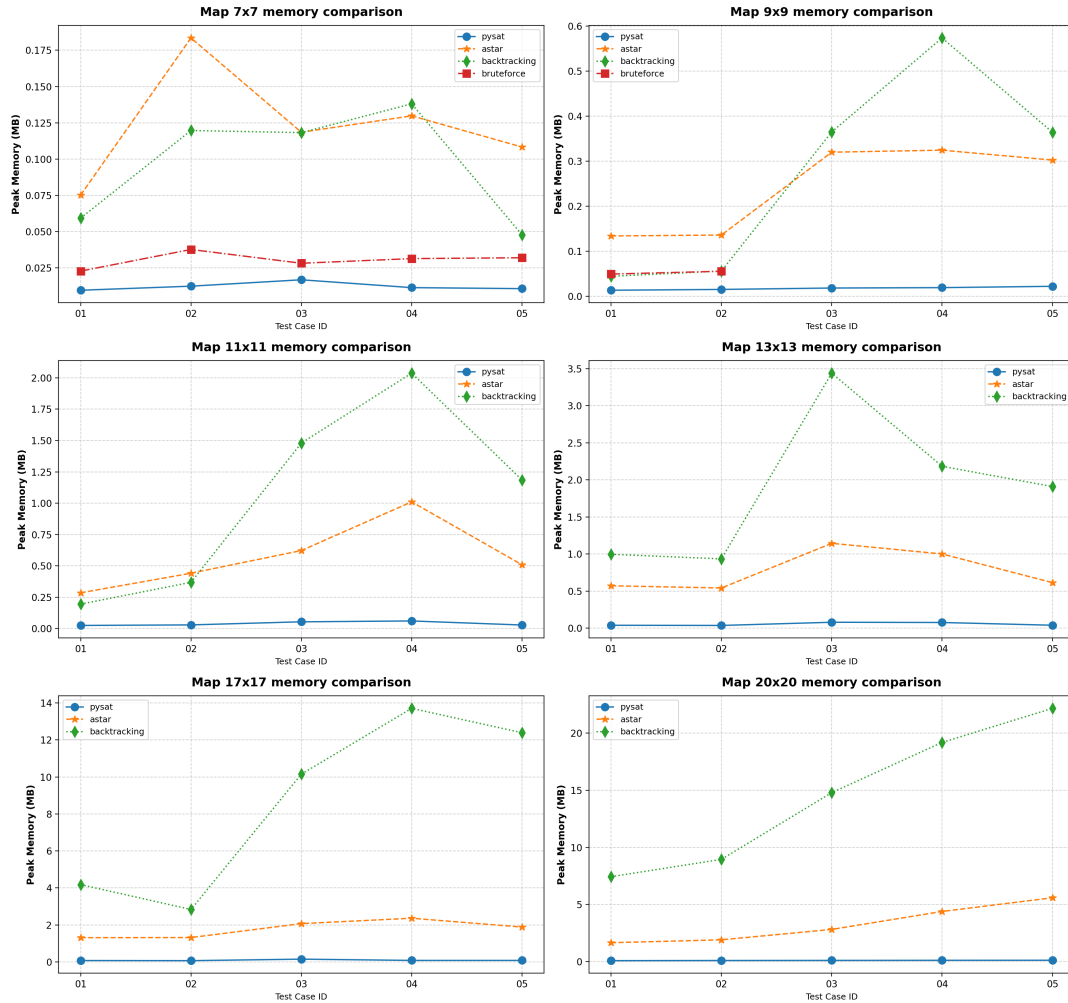
4.2.1 Thời gian thực thi (Execution Time)



Hình 1: Biểu đồ so sánh thời gian thực thi (Trục Y dùng Log Scale)

- **PySAT:** Là thuật toán nhanh nhất một cách áp đảo. Nhờ sử dụng cơ chế *Conflict-Driven Clause Learning (CDCL)*, PySAT giải quyết các Map 20x20 với hơn 6000 mệnh đề chỉ trong chưa đầy 10ms. Đường biểu đồ của PySAT gần như là một đường thẳng nằm sát trục hoành ngay cả khi Map tăng quy mô.
- **Bruteforce:** Gặp hiện tượng bùng nổ tổ hợp ($O(2^n)$). Thuật toán này chỉ khả thi với map 7x7. Ở map 9x9, thời gian chạy vọt lên mức hàng nghìn giây, không thể sử dụng cho thực tế.
- **A* và Backtracking:** Cả hai đều thể hiện tốt trên các Map trung bình. Tuy nhiên, thời gian thực thi của chúng dao động mạnh tùy thuộc vào cấu trúc logic của từng Test Case. Sử dụng **thang đo Log (Log Scale)** giúp quan sát rõ rằng A* thường tốn thời gian hơn Backtracking một chút do chi phí tính toán hàm Heuristic và quản lý hàng đợi ưu tiên.

4.2.2 Bộ nhớ tiêu thụ (Memory Usage)



Hình 2: Biểu đồ so sánh bộ nhớ tiêu thụ

- **Backtracking:** Là thuật toán tốn bộ nhớ nhất (đạt đỉnh 22MB ở Map 20x20). Nguyên nhân do việc duy trì ngăn xếp đệ quy sâu và sao chép các tập mệnh đề trong quá trình lan truyền đơn vị (Unit Propagation).
- **A* Search:** Sử dụng bộ nhớ hiệu quả hơn Backtracking ở quy mô lớn. Thay vì lưu toàn bộ tập mệnh đề, A* chủ yếu lưu trữ trạng thái gán biến (*assignment dict*), giúp giảm tải bộ nhớ xuống chỉ còn khoảng 3-5 MB cho các Map lớn.
- **PySAT:** Cực kỳ tối ưu về bộ nhớ, thường chỉ sử dụng dưới 0.15 MB, cho thấy sự vượt trội của thư viện chuyên dụng.

4.3 Kết luận

Qua thực nghiệm, thuật toán **PySAT** là giải pháp tối ưu nhất về cả thời gian lẫn không gian cho bài toán Hashiwokakero. Thuật toán **A* Search** và **Backtracking** cung cấp góc nhìn về cách áp dụng Heuristic và Unit Propagation để cải thiện tìm kiếm trạng thái, nhưng sẽ gặp giới hạn khi số lượng biến logic vượt quá 500. Thuật toán **Bruteforce** chỉ mang tính chất baseline cho các trường hợp siêu nhỏ.

Tài liệu

- [Crawford and Auton, 1996] Crawford, J. M. and Auton, L. D. (1996). Experimental results on the crossover point in random 3-sat. *Artif. Intell.*, 81(1–2):31–57.
- [Davis et al., 1962] Davis, M., Logemann, G., and Loveland, D. (1962). A machine program for theorem-proving. *Commun. ACM*, 5(7):394–397.
- [Galil and Italiano, 1991] Galil, Z. and Italiano, G. (1991). Data structures and algorithms for disjoint set union problems. *ACM Comput. Surv.*, 23:319–344.
- [Sinz, 2005] Sinz, C. (2005). Towards an optimal cnf encoding of boolean cardinality constraints. In van Beek, P., editor, *Principles and Practice of Constraint Programming - CP 2005*, pages 827–831, Berlin, Heidelberg. Springer Berlin Heidelberg.
- [Stuckey, 2010] Stuckey, P. J. (2010). Lazy clause generation: Combining the power of sat and cp (and mip?) solving. In Lodi, A., Milano, M., and Toth, P., editors, *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, pages 5–9, Berlin, Heidelberg. Springer Berlin Heidelberg.